

```
-- windows function:-  
-- A window function in MySQL is a special type of function that performs  
-- calculations across a set of rows(called a window) related to the  
-- current row.
```

```
create database if not exists windowFunctionsDB;  
use windowFunctionsDB;
```

```
create table baby_names(  
name_id int primary key auto_increment,  
name varchar(225),  
gender enum("Male","Female","Other"),  
total int  
) auto_increment = 500;
```

```
INSERT INTO baby_names (name, gender, total) VALUES
```

```
('Aarav', 'Male', 1200),  
('Anaya', 'Female', 1150),  
('Aadi', 'Other', 300),
```

```
('Vivaan', 'Male', 1200),  
('Aadhya', 'Female', 1080),  
('Sky', 'Other', 280),
```

```
('Aditya', 'Male', 1200),  
('Diya', 'Female', 9300),  
('Sam', 'Other', 280),
```

```
('Krishna', 'Male', 890),  
('Ira', 'Female', 930),  
('Alex', 'Other', 310),
```

```
('Ishaan', 'Male', 870),  
('Myra', 'Female', 930),  
('Robin', 'Other', 310),
```

```
('Arjun', 'Male', 860),  
('Saanvi', 'Female', 1020),  
('Taylor', 'Other', 275),
```

```
('Sai', 'Male', 860),  
('Pari', 'Female', 890);
```

```
select * from baby_names  
order by total desc;
```

```
select *,  
row_number() over(order by total desc) as baby_ranking  
from baby_names;
```

```
select *,
rank() over(order by total desc) as baby_ranking
from baby_names;
```

```
select *,
dense_rank() over(order by total desc) as baby_ranking
from baby_names;
```

```
select *,
row_number() over(order by total desc) as baby_ranking
from baby_names;
```

```
-- 1) row_number():-
-- give each row a rank(no ties)
-- syntax :-
select name, gender, total,
row_number() over(order by total desc) as ranking
from baby_names;
```

```
select name, gender, total,
row_number() over(partition by gender order by total desc) as ranking
from baby_names;
```

```
-- 2) rank():- rank with gaps for ties
select name, gender, total,
rank() over(partition by gender order by total desc) as ranking
from baby_names;
```

```
-- 3) Dense_rank():- ranks without gaps
select name, gender, total,
dense_rank() over(partition by gender order by total desc) as ranking
from baby_names;
```

```
-- 4) Sum():-
select *,
sum(total) over(partition by gender order by total desc) as total_babies
from baby_names;
```

```
-- Find the top 2 most popular baby names for all gender
select * from baby_names;
```

```
select * from
(select *,
dense_rank() over(partition by gender order by total desc) as baby_rank
from baby_names) as temp_baby
where baby_rank < 3;
```

```
-- aggregate functions without group by:-
-- sum(), count(), min(), max(), avg()
select *,
avg(total) over(partition by gender order by total asc) as avg_number_of_babies,
max(total) over(partition by gender order by total asc) as total_babies
from baby_names;
```

```
-- lead() and lag()
-- lead() and lag() functions are used to look at next and previous row.
select name, gender, total,
lead(name) over(partition by gender order by total desc) as "lead"
from baby_names;
```

```
select name, gender, total,
lag(name) over(partition by gender order by total desc) as "lead"
from baby_names;
```

```
select *,
lag(total) over() as "lead"
from baby_names;
```

```
-- ntile(n)
-- splits your table into smaller buckets
select *,
ntile(2) over() as 'partition'
from baby_names;
```

```
select name, gender, total,
case popularity
    when 1 then "Most popular baby name"
    when 2 then "Mid popular baby name"
    else "least popular baby name"
end as "baby_name_popularity"
from
(select *,
```

```
ntile(3) over(partition by gender order by total desc) as "popularity"  
from baby_names) as babY_temp;
```

```
-- first_value(), last_value()  
-- get top/bottom value in the window  
select *,  
last_value(total) over() as "last Value"  
from baby_names  
order by total desc;
```

```
-- cte locks
```